

Name/NetID: Nathan Kariuki / nkari011

Session Date: April 30, 2026 (1-2 hours)

Objective: Set up the structure of the phase 2b folder, implement the lock pool, and build the BFS serial naive.

Attempts made:

The first thing to figure out was why push needs locks when pull does not. In pull each thread writes only to its own vertex so there are no conflicts. In push, a thread processing vertex u writes to all of u 's neighbors, so two threads can hit the same destination at the same time. My first attempt just pushed without any locking which would have caused race conditions in parallel.

For the lock pool, one mutex per vertex seemed clean but with 4.8 million vertices that would be too much memory. The striped pool maps multiple vertices to the same mutex using $v \& (K-1)$.

Solutions/Workarounds:

```
template<int K = 1024>
class LockPool {
public:
    void lock(uint32_t v) { locks[v & (K-1)].lock(); }
    void unlock(uint32_t v) { locks[v & (K-1)].unlock(); }
private:
    mutex locks[K];
};
```

BFS serial naive push scans out-edges and pushes to neighbors:

```
void Process(int tid, uint32_t u, const CsrGraph& g) override {
    if (prev[u] == -1) return;
    for (uint32_t i = g.offsets[u]; i < g.offsets[u+1]; i++) {
        uint32_t v = g.edges[i];
        int candidate = prev[u] + 1;
        if (dist[v] == -1 || candidate < dist[v]) {
            dist[v] = candidate;
            updated[tid] = 1;
        }
    }
}
```

Commits: Phase 2b setup: folder structure, base files, striped lock pool; Add BFS serial naive push implementation

Experiments/Analysis:

Tested on disconnected graph (0 1 1 / 1 2 1 / 3 4 1). Vertices 3 and 4 correctly returned -1 as unreachable from source 0.

Learning outcome:

Push needs locks because any thread can write to any destination vertex. The striped lock pool keeps memory fixed at K mutexes regardless of graph size at the cost of occasional false sharing between unrelated vertices.